



Sport Management Simulation Game

Final Report

Prepared by:

Şilan ATÇI	20230602008
Burak ÇALIŞKAN	20230602017
Beray KAYA	20230602045
Duru Eda CAN	20240602019

Instructor:

Doç. Dr. Kaya OĞUZ

Department of Computer Engineering
CE 216 Project

May 2026

Contents

1	Introduction	3
1.1	Project Summary	3
1.2	Motivation	3
2	System Overview	4
2.1	What the Game Does	4
2.2	Supported Sports	5
3	System Architecture	6
3.1	Layered Architecture	6
3.1.1	UI Layer	7
3.1.2	Handball Plugin – Proof of the Plugin Architecture	8
3.2	Design Principles	8
3.2.1	SOLID Principles	8
3.2.2	Facade Pattern	9
3.2.3	Singleton Pattern	9
4	User Interface Design	10
4.1	UI Overview	10
4.2	Screen Descriptions	10
4.3	UI Architecture	11
4.3.1	SceneManager	11
4.3.2	Controllers	11
4.3.3	Visual Design and User Experience	12
4.3.4	UI Development Challenges	12
5	Application Flow	13
6	Implementation Details	14
6.1	GameFacade	14
6.2	Managers	14
6.2.1	MatchManager	14
6.2.2	TrainingManager	15

6.2.3	LeagueManager	15
6.2.4	SeasonCycleManager	15
6.2.5	PlayerManager	16
6.2.6	TeamManager	16
6.3	Generators	16
6.3.1	PlayerGenerator	16
6.3.2	TeamGenerator	16
6.4	Sport Plugins	16
6.4.1	Football Plugin	16
6.4.2	Handball Plugin	17
6.5	Save and Load System	17
6.5.1	GameState	17
6.5.2	SaveLoadManager	18
7	Technical Details	19
7.1	Tools and Technologies	19
7.2	Project Structure	19
7.3	Module System	20
7.4	Build and Run with Maven	20
7.5	Unit Tests	21
8	Changes from the M1 Design	22
8.1	ITactic Interface and a New ISport Method Were Added to the Sport API	22
8.2	Two Planned Plugin Components Were Consolidated into Core Domain Logic	22
8.3	Single MainWindow Became 13 Controllers	23
8.4	New Application-Layer Classes Were Added	23
8.5	League Configuration Made Dynamic	23
8.6	Three Major Systems Had No Equivalent in the M1 Design	23
8.7	Sport Selection Was Refactored	24
9	Improvements from M2	25
10	Post-Mortem	26
10.1	What Went Right	26
10.2	What Went Wrong	27
11	Future Work	28
12	Conclusion	29

Chapter 1

Introduction

1.1 Project Summary

The Sports Management Simulation Game is a desktop application that lets a single player take the role of a sports team manager. The player chooses a sport (Football or Handball), names a team, builds a squad, and then competes through a full league season against AI-controlled opponents. The game simulates weekly match cycles that include training, match preparation, a live match view with a half-time break, post-match consequences (injuries, energy loss), and end-of-season standings.

The goal of this project was to apply object-oriented design principles, design patterns, and clean software architecture to a problem that is complex enough to show the benefits of these practices. The project was completed in three milestones:

- **M1** – System design: UML class diagrams, use-case analysis, architecture decisions.
- **M2** – Core implementation: Domain layer, Application layer, Sport Plugin layer, unit tests.
- **M3 (this report)** – Complete system: JavaFX user interface, save/load system, second sport (Handball), coach system, and full integration.

1.2 Motivation

This kind of game requires many different responsibilities working together at the same time: player data, match simulation, league standings, season scheduling, and a user-facing interface. Without a good architecture these responsibilities become tangled and very hard to maintain. This project demonstrates how layered architecture and design patterns solve that problem.

Chapter 2

System Overview

2.1 What the Game Does

Sports Manager is a turn-based sports management simulation. The player takes the role of a manager and guides a team through a full league season against AI-controlled opponents. The game supports two sports (Football and Handball), each with its own league size, fixture length, and match simulation rules.

At the start of a new game the player provides a manager name, a team name, a gender category (Male or Female), and a sport type. The application then generates a complete league with AI teams, assigns players and a default coach to every team, and builds a round-robin fixture schedule for the season.

Each in-game week follows a fixed sequence:

1. **Dashboard:** Review the upcoming match, current league standings, coach status, and any active injury reports.
2. **Training:** Choose a training intensity (Light, Medium, or Hard) that affects player condition, energy, and injury risk for the week.
3. **Squad Management:** Adjust the starting lineup and substitute bench before the match.
4. **Coach:** Optionally hire a higher-level coach to improve training multipliers and reduce match energy loss.
5. **Match:** Play through two halves. The simulation engine calculates each period's result from player condition, energy, tactic, and coach bonuses.
6. **Half-Time:** Change the team tactic before the second half begins.
7. **Post-Match:** View the final score, injuries sustained, energy changes, and reputation points earned.

When all match weeks are finished the Season End screen shows the champion and

allows the player to begin the next season with the same squads and updated player states.

2.2 Supported Sports

- **Football:** 18 teams, double round-robin schedule (34 weeks), 2 halves per match.
- **Handball:** 12 teams, double round-robin schedule (22 weeks), 2 halves per match.

Chapter 3

System Architecture

3.1 Layered Architecture

The system is divided into four distinct layers. Each layer has a single responsibility and only communicates with the layer directly below it. The Domain and Application layers were designed and implemented in the previous milestones (M1 and M2). This section focuses on the two areas that were completed in M3: the JavaFX UI layer and the Handball sport plugin, which together demonstrate that the architecture works exactly as intended.

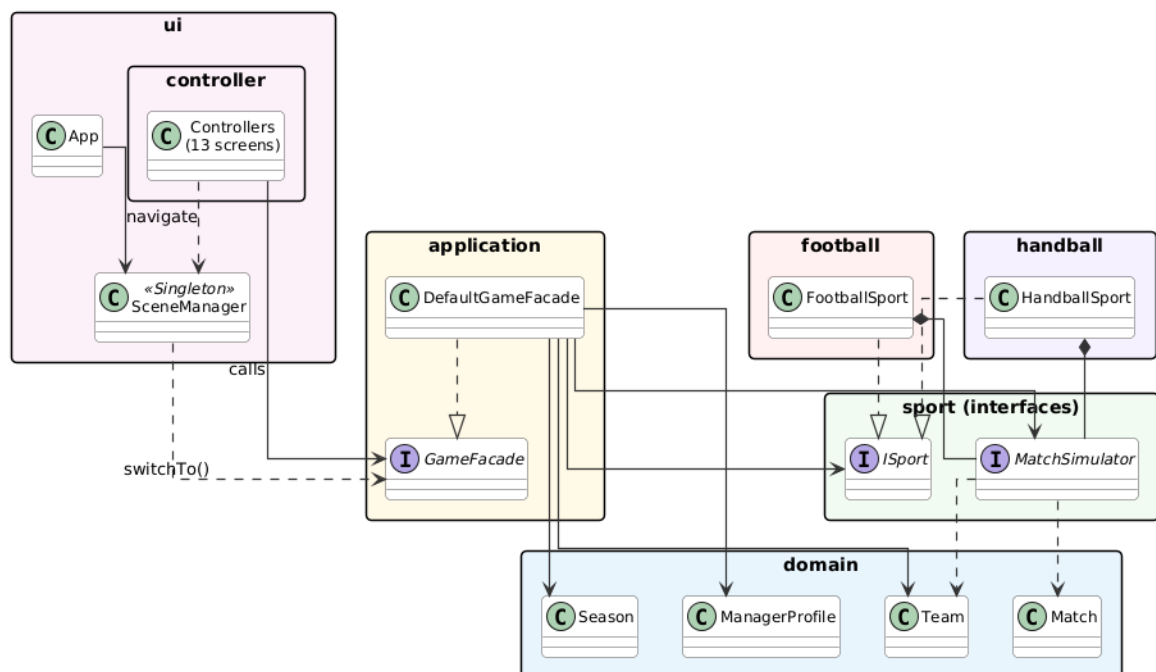


Figure 3.1: Four-layer architecture of the Sports Management Simulation Game

3.1.1 UI Layer

Package: `ui`, `ui.controller`

The UI layer is the main new component of M3. It is built entirely with JavaFX and sits on top of the existing application layer without modifying any of it. The layer has three main parts:

- **App:** The JavaFX entry point. It creates the primary Stage, passes it to SceneManager, and navigates to the main menu to start the application.
- **SceneManager:** A Singleton that owns the Stage reference and handles all screen transitions. Controllers call `switchTo("screen-name", facade)` and the manager replaces the root of the existing Scene without reopening the window. Full details are in Section 4.3.1.
- **SoundManager:** A Singleton that loads ".m4a" audio files from the resources folder and plays them at the right moments (button clicks, victory, draw, game-over, champion fanfare).

Each screen has a dedicated controller class in the `ui.controller` package. A controller receives a GameFacade in its constructor and builds its layout programmatically inside `getRoot()`. Controllers may read domain data objects (such as Player, Match, and StandingEntry) for display purposes, but all business operations are delegated through the GameFacade interface. Controllers never reference any Manager or Generator class directly, keeping business logic out of the UI layer.

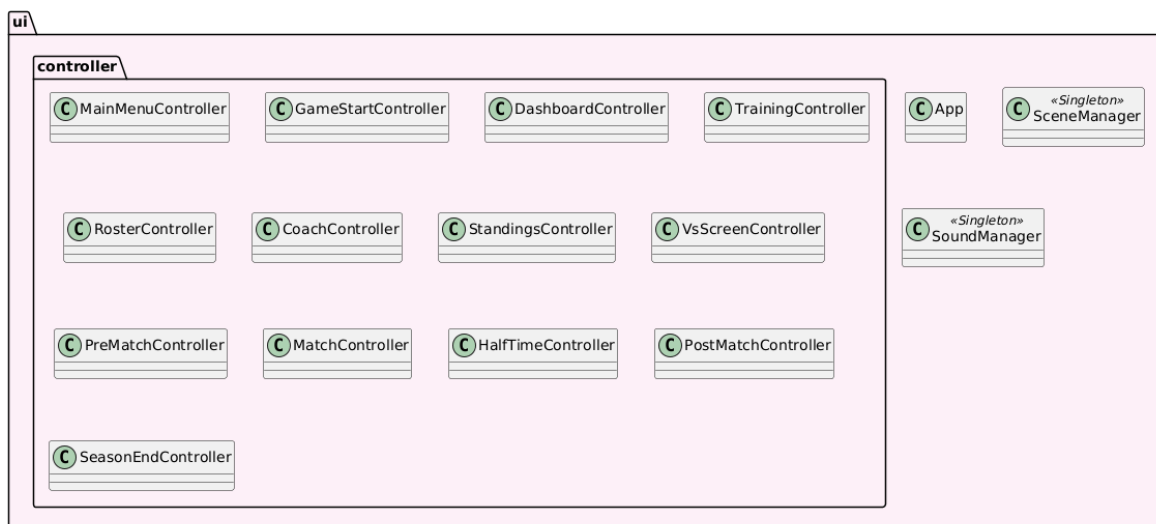


Figure 3.2: UI-layer architecture of the Sports Management Simulation Game

3.1.2 Handball Plugin – Proof of the Plugin Architecture

Package: handball

Adding Handball was the clearest test of whether the plugin architecture actually works. A new handball package was created with seven classes implementing the sport interfaces defined in the sport package. No existing business logic or UI code needed to change; only a single entry was added to the sport registry in DefaultGameFacade to register the new sport. The full class breakdown is covered in Section 6.4.

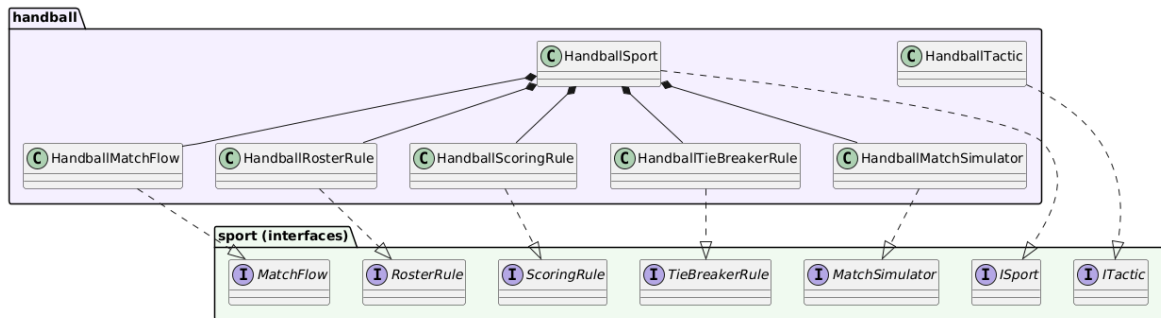


Figure 3.3: Handball plugin class structure

3.2 Design Principles

3.2.1 SOLID Principles

- **Single Responsibility Principle (SRP):** Each class has one clearly defined job. For example, TrainingManager only applies training effects, MatchManager only manages match simulation, and PlayerGenerator only creates player objects. No class handles multiple unrelated responsibilities.
- **Open/Closed Principle (OCP):** The system is open for extension but closed for modification. Adding a new sport requires creating a new package with implementations of the ISport interfaces, and registering it with one line in the SPORT_REGISTRY map inside DefaultGameFacade. No other existing class needs to be changed: the UI sport selection, startNewGame, and the save/load system all pick up the new entry automatically.
- **Liskov Substitution Principle (LSP):** Any ISport implementation can replace another without changing the callers. MatchManager and DefaultGameFacade hold only an ISport reference, so switching from Football to Handball at game setup silently changes league size, roster rules, and match simulation with no conditional logic in the caller.

- **Interface Segregation Principle (ISP):** The plugin layer is split into several small interfaces (MatchFlow, MatchSimulator, RosterRule, ScoringRule, TieBreakerRule) instead of one large interface. Each class only depends on the parts it actually uses.
- **Dependency Inversion Principle (DIP):** The DefaultGameFacade depends on the ISport interface, not on concrete sport classes. The UI depends on the GameFacade interface, not on DefaultGameFacade directly.

3.2.2 Facade Pattern

The `GameFacade` interface provides a single, simple entry point for the entire game system. The UI layer calls methods like `startNewGame()`, `applyTraining()`, `playPeriod()`, and `submitWeekResults()` without knowing about the managers, generators, or domain objects behind them. This hides the complexity of the application layer and makes the UI code simple and clean.

3.2.3 Singleton Pattern

The `SceneManager` and `SoundManager` classes use the Singleton pattern. Only one instance of each is created for the lifetime of the application. This ensures that all controllers access the same stage and the same audio engine.

Chapter 4

User Interface Design

4.1 UI Overview

The user interface is built entirely in Java using the JavaFX framework. All layouts are constructed programmatically in controller classes. A single CSS file (style.css) in the resources folder provides the visual theme for all screens. The theme uses a dark background with green, blue, and orange accent colours.

4.2 Screen Descriptions

For a detailed walkthrough of all 13 screens refer to the accompanying User Manual. Table 4.1 lists each screen together with its controller class and primary purpose.

Table 4.1: Screens and their corresponding controller classes

Screen	Controller Class	Purpose
Main Menu	MainMenuController	Start a new game or load from one of three save slots
New Game	GameStartController	Enter manager name, team name, gender, and sport
Dashboard	DashboardController	Weekly hub: upcoming match, standings, coach and injury cards
Training	TrainingController	Choose training intensity (Light / Medium / Hard)
VS	VsController	Full-screen match announcement before each game
Pre-Match	PreMatchController	Squad and tactic review before kick-off
Match	MatchController	Period-by-period play, live score, player status bars
Half-Time	HalfTimeController	Tactical changes between the two halves
Post-Match	PostMatchController	Final result, injuries, energy changes, reputation gained
Standings	StandingsController	Full league table with user team highlighted
Squad Management	RosterController	Move players between starting lineup and substitutes
Coach	CoachController	Hire coaches unlocked by season number and reputation
Season End	SeasonEndController	Champion display, final standings, start next season

4.3 UI Architecture

4.3.1 SceneManager

The SceneManager is a Singleton that holds a reference to the JavaFX Stage. All screen transitions go through its `switchTo(String screenName, GameFacade facade)` method. The method uses a switch statement to construct the correct controller and calls `getRoot()` to get the Parent node. It then replaces the root of the existing Scene rather than creating a new window. This keeps the window open and the CSS stylesheet attached.

```
1 public void switchTo(String screenName, GameFacade facade) {
2     Parent root;
3     switch (screenName) {
4         case "dashboard":
5             root = new DashboardController(facade).getRoot();
6             break;
7         case "match":
8             root = new MatchController(facade).getRoot();
9             break;
10        // ... other cases
11        default:
12            throw new IllegalArgumentException("Unknown screen:
13        "
14            + screenName);
15    }
16    stage.setScene().setRoot(root);
17    stage.show();
18 }
```

Listing 4.1: SceneManager switchTo method (excerpt)

4.3.2 Controllers

Each screen has a dedicated controller class in the `ui.controller` package. A controller receives a GameFacade in its constructor. Its only public method is `getRoot()`, which builds the entire layout programmatically and returns it. This keeps each controller self-contained and easy to test or replace independently.

4.3.3 Visual Design and User Experience

The interface uses a consistent dark theme throughout, with green, blue, and orange accent colours applied semantically. Green indicates positive states (standings highlight, win feedback), orange flags warnings (low energy, upcoming match), and red marks critical conditions (injured players). This colour coding allows the player to assess squad status at a glance without reading individual values.

Injured players are highlighted with a distinct badge on the Match and Squad screens, and a dedicated injury card appears on the Dashboard whenever any squad member is injured. Sound feedback reinforces game events: a click sound confirms every button press, and distinct audio cues play for match victory, draw, game-over, and season champion outcomes. Together these choices give the interface a responsive, game-like feel without relying on animation.

4.3.4 UI Development Challenges

Building the interface revealed several engineering problems that were not apparent from the architecture design alone.

Scene transitions and state synchronisation. Each screen is rebuilt when the player navigates to it. Any state not persisted in the facade is lost on transition. This caused the Dashboard to call the current-match accessor before the week had been started, crashing the scene transition. The fix required adding a separate read-only facade method so the Dashboard could display upcoming match data without advancing the game state.

Layout stability with dynamic content. ComboBox controls on the New Game screen caused adjacent panels to resize when the dropdown opened, because HBox distributes leftover space based on preferred widths. Switching to a GridPane with percent-width column constraints fixed each card's width to a share of available space regardless of child preferred sizes, eliminating the layout shift.

Stateless controller communication. Because controllers are stateless and communicate only through the facade, passing transient data between screens (such as the match result from the Match screen to the Post-Match screen) must be routed through the facade or stored in application state. This constraint enforced the architectural boundary and prevented direct inter-controller dependencies.

Chapter 5

Application Flow

The overall game follows a clear lifecycle from start to end.

1. **Game Setup:** The player provides their manager name, team name, gender, and sport. `DefaultGameFacade.startNewGame()` creates the league, AI teams, and fixture, then starts Season 1.
2. **Weekly Cycle (repeated for each week):** Each week the player goes through training, squad review, and the match. After the match, results are submitted, standings are updated, and the week counter advances.
3. **Season End:** When all weeks are finished, `isSeasonComplete()` returns true. The Season End screen is shown with the champion. The player can call `startNewSeason()` to begin the next season with the same teams and updated player states.
4. **Save / Load:** At any point from the Dashboard the player can save to a slot (1, 2, or 3). The entire game state is serialised to a JSON file using Gson. Loading restores the state including standings, week number, player attributes, coach, and tactic.

Chapter 6

Implementation Details

6.1 GameFacade

GameFacade is a Java interface that defines all operations the UI is allowed to perform. It is implemented by DefaultGameFacade. The interface is divided into logical sections:

- Sport discovery (`getAvailableSportNames`) returns the list of registered sport names so the UI can populate its selection menu without knowing any concrete sport class
- Initialisation (`startNewGame`)
- Profile and team queries
- Season lifecycle (`startNewSeason`, `isSeasonComplete`)
- Weekly cycle (`startWeek`, `applyTraining`, `playPeriod`, `submitWeekResults`)
- Standings and week queries
- Team management (`addPlayerToStarting`, `setTactic`, `setCoach`)
- Save and load

This design makes it possible to replace `DefaultGameFacade` with a different implementation (for example, a networked version or a test stub) without changing a single line of UI code.

6.2 Managers

6.2.1 MatchManager

`MatchManager` orchestrates all match-related operations. Its main responsibilities are:

- **simulateAIMatches:** Iterates over all matches in a `MatchWeek`, skips the user's match, and calls `matchSimulator.simulateMatch()` for each AI match.

- **playUserPeriod:** Delegates a single period to the sport's `MatchSimulator.playPeriod()` method.
- **applyPostMatchEffects:** After a match it applies condition changes (win +3, draw -2, loss -8, base wear -5), energy loss (starters -15, substitutes -5, reduced by coach), injury risk increase (+3 to +6 per match), random injuries for high-risk players, and one step of injury recovery.

6.2.2 TrainingManager

`TrainingManager` applies training and weekly recovery to a team's players. Training effects are shown in Table 6.1. A coach multiplier improves the condition gain and reduces the energy loss based on the current coach-team relationship value (0–100).

Table 6.1: Training intensity effects (before coach multiplier)

Intensity	Energy Loss	Condition Gain	Injury Risk
Light	-5	+3	-2
Medium	-10	+6	0
Hard	-20	+10	+5

Weekly recovery restores energy before each week: healthy starters recover 20 points, healthy substitutes recover 25 points, and injured players recover 10 points. Recovery also reduces injury risk.

6.2.3 LeagueManager

`LeagueManager` creates a `League` object that contains the user's team and the required number of AI teams. It uses `TeamGenerator` to produce AI teams with random names and generated rosters. It also supports restoring a league from saved AI team names during a load operation.

6.2.4 SeasonCycleManager

`SeasonCycleManager` keeps track of which season is active, which week is current, and the full list of all past seasons. It delegates season creation, fixture access, and standings queries to the `Season` and `LeagueTable` domain objects. It also provides `forceCompleteAndAdvance()` which is used during game loading when a completed season was saved.

6.2.5 PlayerManager

PlayerManager is the single place where player objects are created and stored. It assigns a unique auto-incremented ID to every player. Other classes call `playerManager.createPlayer()` rather than calling the Player constructor directly.

6.2.6 TeamManager

TeamManager manages changes to the user's team. It validates roster changes (e.g. preventing duplicates), delegates to the Team domain object, and sets the tactic and coach on the team.

6.3 Generators

6.3.1 PlayerGenerator

PlayerGenerator creates Player objects with randomised attributes. Names are selected from two static pools in NamePool: one for male names and one for female names. Age is random between 18 and 50. Energy, condition, and injury risk are randomised within predefined ranges.

6.3.2 TeamGenerator

TeamGenerator creates Team objects for AI opponents. It generates a team with a random name and fills it with generated players of the correct gender using PlayerGenerator.

6.4 Sport Plugins

6.4.1 Football Plugin

The FootballSport class assembles all football-specific components:

- **FootballMatchFlow**: Defines 2 periods per match.
- **FootballRosterRule**: Defines valid roster size (11 starters).
- **FootballScoringRule**: Calculates goals based on team strengths and random factors.
- **FootballTieBreakerRule**: Uses goal difference and then goals scored to break ties.

- **FootballMatchSimulator**: Orchestrates the full match using the above components.
- **FootballTactic**: Provides ATTACKING, BALANCED, and DEFENSIVE tactic options.
- 18 teams, double round-robin (34 match weeks).

6.4.2 Handball Plugin

The HandballSport class follows exactly the same structure with handball-specific rules:

- **HandballMatchFlow**: Defines 2 periods per match.
- **HandballRosterRule**: Defines valid roster size (7 starters).
- **HandballScoringRule**: Produces higher-scoring results typical of handball.
- **HandballTieBreakerRule**: Handles tie-breaking for the handball league.
- **HandballMatchSimulator**: Orchestrates handball matches.
- 12 teams, double round-robin (22 match weeks).

Adding a third sport would only require creating a new package that implements the same six interfaces. No existing code would need to change.

6.5 Save and Load System

6.5.1 GameState

GameState is a plain Java object that captures every piece of information needed to restore a game. It contains:

- Manager name, reputation, season number.
- Sport name, gender, team name, coach relationship.
- Coach details (name, level, required season, required reputation).
- Tactic style (as a string name of the **PlayStyle** enum).
- Player data for starters and substitutes (name, age, energy, condition, injury risk, injury status, games remaining injured).
- AI team names.
- Current week number, total weeks.
- Full standings data for every team.

- Season complete flag.
- Save date and time.

6.5.2 SaveLoadManager

SaveLoadManager uses the Gson library to serialise GameState to JSON and write it to a file named `save_slot_{id}.json`. Loading reads the file, deserialises it, and returns the GameState. It also reads all three slot files to produce a list of slot summaries (save date or "Empty") for the UI to display.

The `DefaultGameFacade.restoreFromGameState()` method takes the loaded state and reconstructs all managers, teams, and standings from it, effectively restoring the game exactly as it was when saved.

Chapter 7

Technical Details

7.1 Tools and Technologies

Table 7.1: Technologies used in the project

Technology	Version	Purpose
Java	26	Core programming language
JavaFX	26	Desktop user interface framework
Maven	3.x	Build tool and dependency management
JUnit 5	5.10.2	Unit testing framework
Gson	2.13.2	JSON serialisation for save/load
IntelliJ IDEA	Latest	Development environment

7.2 Project Structure

```
src/  
  main/  
    java/  
      application/ - GameFacade, managers, generators, save/load  
      domain/      - Core business objects (Player, Team, Season, ...)  
      football/    - Football plugin implementation  
      handball/    - Handball plugin implementation  
      sport/       - Plugin interfaces (ISport, MatchSimulator, ...)  
      ui/          - App, SceneManager, SoundManager  
      ui/controller/ - One controller per screen  
    resources/  
      style.css    - Global CSS theme  
      images/      - Image assets (vs.jpg)  
      sounds/      - Audio files (.m4a)  
  test/  
    java/
```

application/	- MatchManagerTest, TrainingManagerTest
domain/	- PlayerTest, TeamTest, CoachTest
football/	- FootballScoringRuleTest
handball/	- HandballScoringRuleTest, HandballMatchSimulatorTest, ...

7.3 Module System

The project uses the Java module system (`module-info.java`). It declares requires directives for `javafx.controls`, `javafx.fxml`, `javafx.media`, and `com.google.gson`. Five packages are opened to Gson for JSON reflection during save/load operations: `application`, `domain`, `football`, `handball`, and `sport`. This is necessary because `GameState` stores player and team data whose classes live in the domain package.

7.4 Build and Run with Maven

All build and run operations are done through Apache Maven from the project root directory.

```
//Compile all source files
mvn compile

//Run all JUnit 5 tests
mvn test

//Launch the JavaFX application
mvn javafx:run
```

7.5 Unit Tests

The test suite covers the most important business logic components. Tests are written with JUnit 5 and follow the AAA pattern (Arrange, Act, Assert).

Testing was particularly valuable in this project because much of the game logic, including energy loss, condition clamping, injury probability, and training multipliers, involves numeric calculations with subtle boundary conditions that are difficult to verify through manual gameplay alone. Unit tests made these invariants explicit and caught several incorrect values before the UI was available. Second, the plugin architecture required verifying that each sport implementation satisfied the same behavioural contracts. The Handball test classes confirmed that roster validation, scoring ranges, and match simulation all behaved correctly before any Handball screen was built.

Table 7.2: Test classes and what they test

Test Class	What is tested
PlayerTest	Player attribute clamping, injury logic, recovery
TeamTest	Roster management, squad validation
CoachTest	Coach unlock conditions, multiplier calculations
LeagueManagerTest	League creation, team count, user team assignment
MatchManagerTest	Energy loss, injury risk, condition change
TrainingManagerTest	Training effects, weekly recovery
FixtureTest	Fixture generation, match week structure
LeagueTableTest	Standings update, sorting, tie-breaking
FootballScoringRuleTest	Score generation with different team strengths
FootballTieBreakerRuleTest	Football standings tie-breaking logic
HandballScoringRuleTest	Handball scoring characteristics
HandballRosterRuleTest	Handball roster validation (7 starters)
HandballMatchSimulatorTest	Full handball match simulation

Chapter 8

Changes from the M1 Design

The M1 document established the four-layer architecture, the sport plugin concept, the GameFacade as the single UI boundary, and the core domain entities. These decisions remained stable throughout the project. However, several things changed or were added during implementation that were not part of the original design.

8.1 ITactic Interface and a New ISport Method Were Added to the Sport API

The M1 design already defined six Sport API interfaces (ISport, MatchSimulator, MatchFlow, RosterRule, ScoringRule, TieBreakerRule), which remained stable throughout implementation. Two things were added that were not in M1. First, a seventh interface ITactic was introduced to represent the play style a team can use during a match. This was necessary because Football and Handball have different tactic options and the application layer cannot reference sport-specific tactic classes. Second, a getDefaultTactic() method was added to ISport so that LeagueManager could assign each newly created team the correct default tactic for its sport without any hardcoded dependency on a concrete tactic class.

The M1 text also described ScoringRule as determining how many points are given for a win, draw, or loss. During implementation the responsibility was clarified: ScoringRule generates the goals scored in a match, while league points (3 for a win, 1 for a draw, 0 for a loss) are calculated by LeagueTable and are the same for every sport.

8.2 Two Planned Plugin Components Were Consolidated into Core Domain Logic

The M1 Sport API diagram showed InjurySystem and SubstitutionManager as separate classes inside the Football plugin. In the final system, injury logic lives inside MatchManager and the Player class, and substitutions are handled directly through the Team

class. Extracting these into separate plugin components would have added interface complexity without any architectural benefit at this project scope.

8.3 Single MainWindow Became 13 Controllers

The M1 architecture diagram showed a single MainWindow class as the entire UI layer. The final implementation has 13 dedicated controller classes, a SceneManager singleton, a SoundManager singleton, and an App entry point. The scale of the UI was considerably larger than the M1 placeholder suggested.

8.4 New Application-Layer Classes Were Added

The M1 design included LeagueManager, TeamManager, and MatchManager, but did not include PlayerManager, PlayerGenerator, TeamGenerator, TrainingManager, or SeasonCycleManager. These were added to keep responsibilities properly separated. PlayerManager became especially important for the save/load system, which required every player to have a stable unique identifier across save and load cycles.

8.5 League Configuration Made Dynamic

The M1 domain model showed that a Season could contain multiple leagues. In the final implementation there is exactly one league per season. Additionally, league size and fixture length were initially hardcoded for football. This was refactored so that the team count and schedule length are determined by the sport itself, which is what made the 12-team, 22-week handball league work correctly without any special casing.

8.6 Three Major Systems Had No Equivalent in the M1 Design

The save/load system, the coach progression system, and the manager reputation system had no equivalent in the M1 design. All three were introduced during M3. Their details and the bugs encountered during implementation are described in the Improvements from M2 and Post-Mortem chapters.

8.7 Sport Selection Was Refactored

The original `startNewGame()` method was designed to accept an `ISport` object directly, which required the UI layer to instantiate concrete sport classes. This violated the layered architecture principle that M1 itself defined. During M3 the method signature was changed to accept a sport name string, and a central sport registry was introduced in `DefaultGameFacade` so that all sport instantiation is centralised. Adding a new sport now requires only one line in the registry; the UI dropdown and the save/load restore path both update automatically.

Chapter 9

Improvements from M2

M3 completed the system by adding the following features that were either missing or incomplete in M2:

- **Complete JavaFX UI:** All 13 screens with full interaction, navigation, and dark-themed CSS styling.
- **Handball Plugin:** The full handball sport package implementing all seven sport interfaces. M2 only had the Football plugin.
- **Save/Load System:** A GameState snapshot class and a SaveLoadManager serialise the full game state to a JSON file using Gson. Loading restores standings, player attributes, coach data, week number, and tactic. Three save slots are available from the Dashboard.
- **Coach System:** Five coaches with level-based unlock requirements tied to manager reputation and season number. Each coach provides a training multiplier and reduces match energy loss. Bonus effectiveness is scaled by a coach-team relationship value (0–100) that grows with wins and shrinks with losses.
- **Manager Reputation:** Reputation points are awarded after each match (win: +4, draw: +1) and accumulate across seasons. They serve as the unlock currency for higher-level coaches.
- **Sound System:** Audio feedback is played at appropriate moments, including button clicks, match victories, draws, game-over events, and season champion fanfares.
- **Injury Report on Dashboard:** A dedicated injury card appears on the Dashboard whenever any squad member is injured, so the player is aware before selecting the lineup.
- **Post-Match Condition System:** Player condition now changes after every match based on the result: a win gives a small bonus, while a loss applies a larger penalty, in addition to the weekly training effect.

Chapter 10

Post-Mortem

10.1 What Went Right

- **The plugin architecture worked exactly as designed.** Adding the Handball sport plugin required zero changes to the domain, application, or UI layers. Only a new package with seven classes was needed. This was the main architectural goal of M1 and it was fully met.
- **The GameFacade boundary enabled parallel development.** Different team members worked on UI screens and backend systems simultaneously without conflicts. The facade interface defined a clear contract: UI controllers only needed to know the facade methods, not the internals of the managers.
- **Domain entities were stable from M1.** Core classes such as Player, Team, League, LeagueTable, Fixture, and Match required almost no structural changes from the original design. The early modelling investment paid off throughout all three milestones.
- **Unit tests caught real bugs before the UI was available.** The JUnit 5 test suite identified a critical match lifecycle ordering issue, incorrect condition clamping, and a duplicate state update that would have been very difficult to reproduce through manual UI testing alone.
- **Dynamic league configuration generalised cleanly.** When Handball was added, neither the league creation nor the fixture logic required any changes. Both the 18-team and 12-team formats worked from the same code path, confirming that delegating configuration to the sport plugin was the right design decision.

10.2 What Went Wrong

- **The save/load system required four separate bug fixes.** Because save/load was not designed in M1, the GameState class was built without a full picture of what needed to be captured. This led to a series of bugs: loading always reset to Week 1, standings were lost after load because AI team names were not persisted, and injury state was not correctly reconstructed on seasonal loads. Each fix required adding new fields or methods to already finished classes.
- **Duplicate state updates caused subtle gameplay bugs.** The weekly energy recovery was called in two places: inside DefaultGameFacade and again in the post-match screen. This caused players to recover twice per week, making the game too easy. Similarly, the condition change was applied twice to substitutes. Both bugs were caused by responsibilities not being clearly separated between layers, leading the same logic to be applied in multiple places.
- **The dashboard crashed when opened before a week had started.** The method for retrieving the current match was called before the week start logic had run, throwing an exception that crashed the scene transition. The fix required adding a separate read only method to the facade so the dashboard could safely display the upcoming match without triggering state changes.
- **Starting a new game in the same session crashed the application.** The league manager threw an exception when the player returned to the main menu and started a new game without restarting the application. Session reuse was never considered in the M1 design, where a single game lifetime per launch was assumed.
- **Handball rosters produced duplicate player names.** The name pool was originally sized for a single 18-player football squad. Drawing names for a separate handball team could produce overlapping names. The fix required generating all player names from a single shuffled pool at once, rather than drawing independently per team.
- **Game balance required three full rebalancing iterations.** Coach bonuses, reputation gain, injury risk, energy loss, and score ranges all needed adjustment after the full game loop became playable. No balance targets were set during M1 or M2. Starting balance design earlier would have reduced the number of late stage fixes.

Chapter 11

Future Work

- **Transfer market:** Allow players to be signed and released between seasons, adding a budget management mechanic to the game.
- **Player progression:** Add experience points and skill levels so players improve or decline with age over multiple seasons.
- **Animated match view:** Show a visual representation of the match as events happen (shots, fouls, substitutions) instead of an immediate result.
- **AI difficulty levels:** Give AI teams different strength profiles and tactical tendencies to vary the challenge.
- **Additional sports:** The plugin architecture is designed for exactly this. Basketball, Volleyball, or Ice Hockey could be added as new sport packages with a single entry added to the `SPORT_REGISTRY` map in `DefaultGameFacade`; the UI sport selection dropdown and save/load restore path update automatically.
- **Tournament mode:** Add a cup competition that runs alongside the league season.
- **Statistics screen:** Show career statistics for the manager and individual players across multiple seasons.
- **Responsive UI:** Make the window resizable using `VBox.setVgrow` and `HBox.setHgrow` constraints so the layout adapts to different screen sizes.
- **Database persistence:** Replace JSON file saves with a local SQLite database for more robust and relational storage.
- **Localisation:** Add internationalisation support so the game can be played in different languages.

Chapter 12

Conclusion

This project shows how a moderately complex application can be built cleanly using object-oriented principles and a layered architecture. The four-layer design (Domain, Application, Plugin, UI) kept each part of the system focused on a single job. The Facade pattern made the UI code simple and independent of the business logic. The Plugin Architecture made it straightforward to support two sports and would allow more to be added with minimal effort.

M3 brought the project from a working engine (M2) to a fully playable game with a complete dark-themed JavaFX interface, sound feedback, a coach-and-reputation progression system, a save/load feature, and a second sport. All of the key architectural decisions made in M1 held up well during implementation. The specific deviations from the original The result is a working sports management simulation that demonstrates a range of software engineering concepts: SOLID principles, Separation of Concerns, the Facade and Singleton design patterns, a plugin-based extensibility model, period-based simulation, persistent state management, and test-driven development for core business logic.

The result is a working sports management simulation that demonstrates a range of software engineering concepts: SOLID principles, Separation of Concerns, the Facade and Singleton design patterns, a plugin based extensibility model, period based simulation, persistent state management, and test driven development for core business logic.